Problemset 2 AMAS

Asger Thormann

February 25, 2026

Loading the data

I use pandas to load the .csv file. When loading the .csv file I get an error of too many values in line 151. It seems like there was put an extra comma, i'll just delete the comma, I provide the datafile I use. When converting the times to seconds I find that one value is not a numeric and I convert the accidental o to a 0.

1

1.1

I fit the the function given in the problem set with parameters μ, σ and ξ. I use both my own home cooked ln likelihood function and the library function minuit.cost.UnbinnedNLL which i minimize with minuit. I obtain the same parameters for both methods:

$$\mu = (2.96 \pm 0.07) \cdot 10^3$$
$$\sigma = (6.1 \pm 0.5) \cdot 10^2$$
$$\xi = 0.22 \pm 0.08$$

Errors on the parameters are given by the minuit fitting method.

1.2

The negative ln-likelihood is $-LLH = 1560.04$

1.3

Here the fit is scaled to the histogram. The range is $x = (2000, 5000)$ with 30 bins giving a bin width of 100 seconds. This plot is shown in figure 1.

1.4

Using scipy.integrate and finding Tanyusha, SomethingAboutGoats' time (3747 s). I calculate the value fraction as:

$$f_{slow} = 1 - \int_0^{t_{tanyusha}} GEV(x, \mu, \sigma, \xi) = 0.1985$$

1.5

Integrating the dataset (by summing the counts) I find that the fraction is 0.2273.

1.6

I calculate the error on the fraction using the formula $\sigma_f = \sqrt{\frac{f(1-f)}{n}}$ and then I find the number of standard deviations using:

$$\frac{\Delta f}{\sqrt{\sigma_{data}^2 + \sigma_{fit}^2}} = 0.70\sigma$$

This shows that the numbers are within 1 standard deviation meaning that they are in agreement. This is under the assumption tha the fraction are gaussianly distributed.

1.7

In figure 2 I show the scaled CDF to the dataset. I create the CDF by integrating the GEV over the entire range.

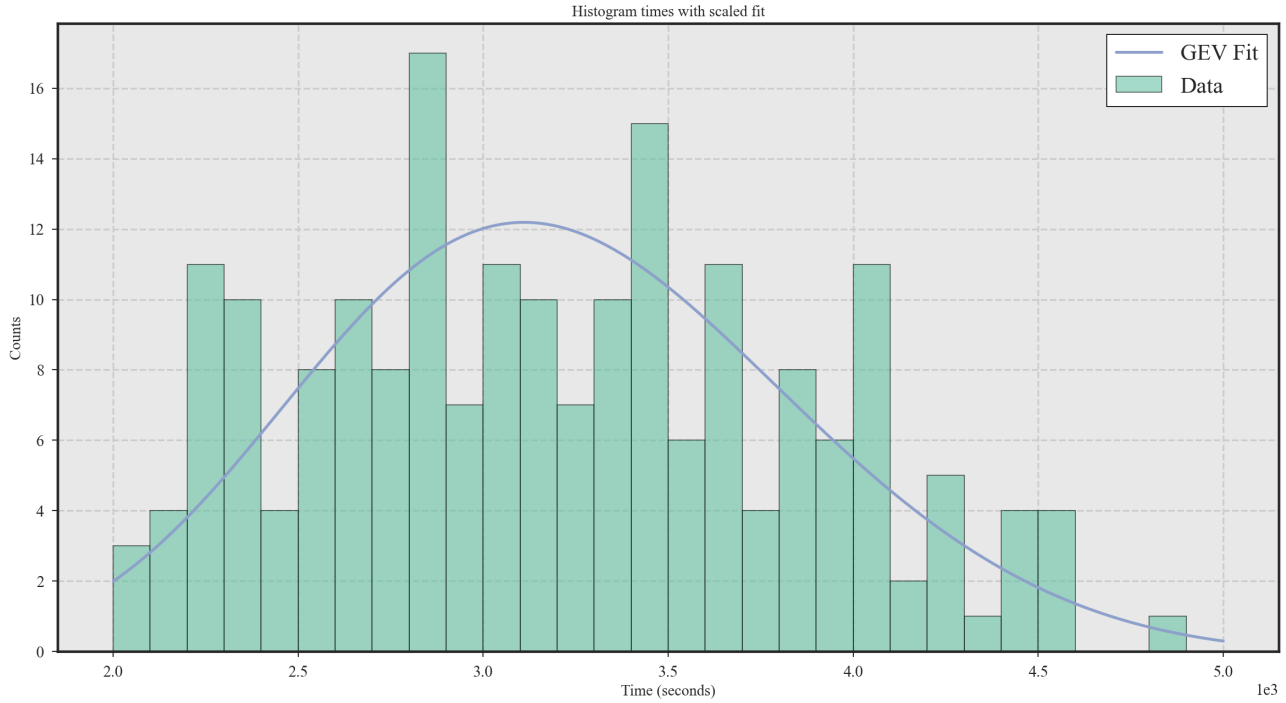

Figure 1: Histogram and fit. Fit is scaled to histogram.

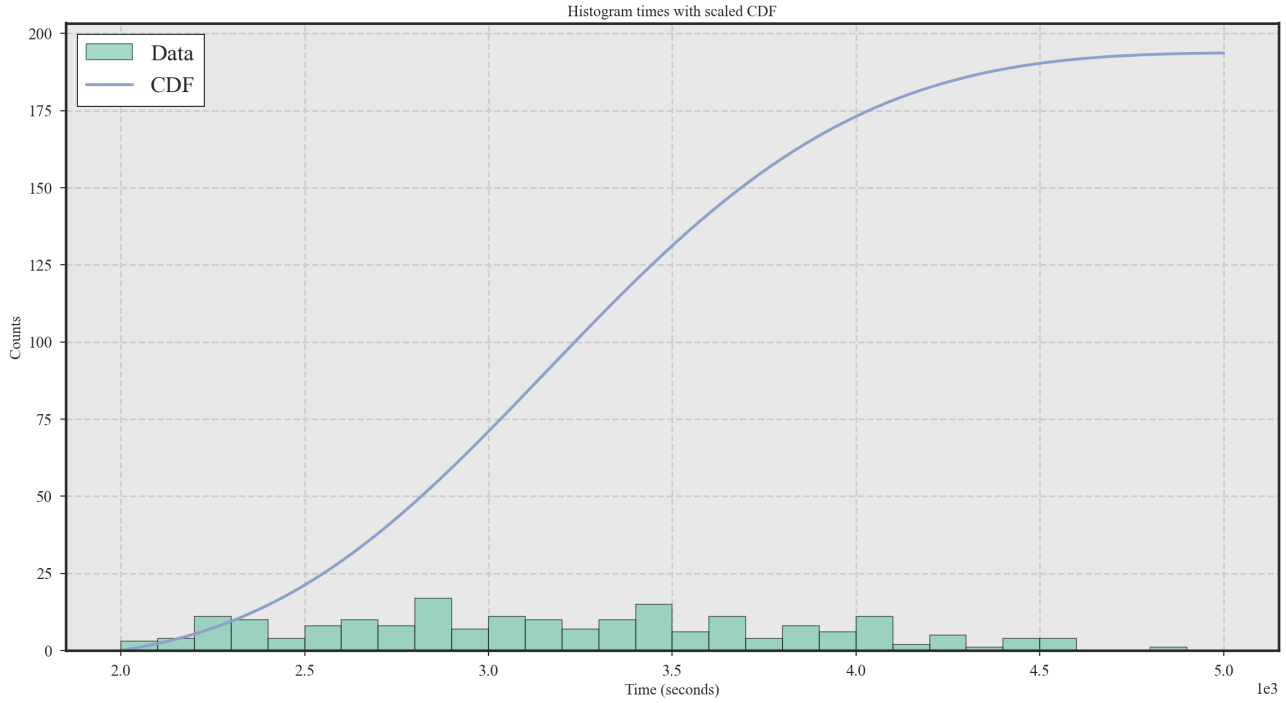

Figure 2: CDF scaled to the counts of the histogram

1.8

Calculating the probability of observing a faster time than the fastest in the dataset I integrate the GEV using the given parameters from 0 to the minimum. I find the probability:

$$P(t < t_{min}) = 4.8 \cdot 10^{-4}$$

2

2.1

I write up the functions of the likelihood and the prior and normalize them over the range $x = [0, 4]$. For the likelihood I integrate using scipy.integrate.quad() and thus derive the normalization constant. For the prior I integrate analytically using the given intervals and thus finding: $N = 1.6 + 2.66 \cdot 0.5 = 2.93$.

I normalize the posterior using scipy.integrate.quad() on the product of the likelihood and the posterior finding the normalization constant. Figure 3 shows all three functions.

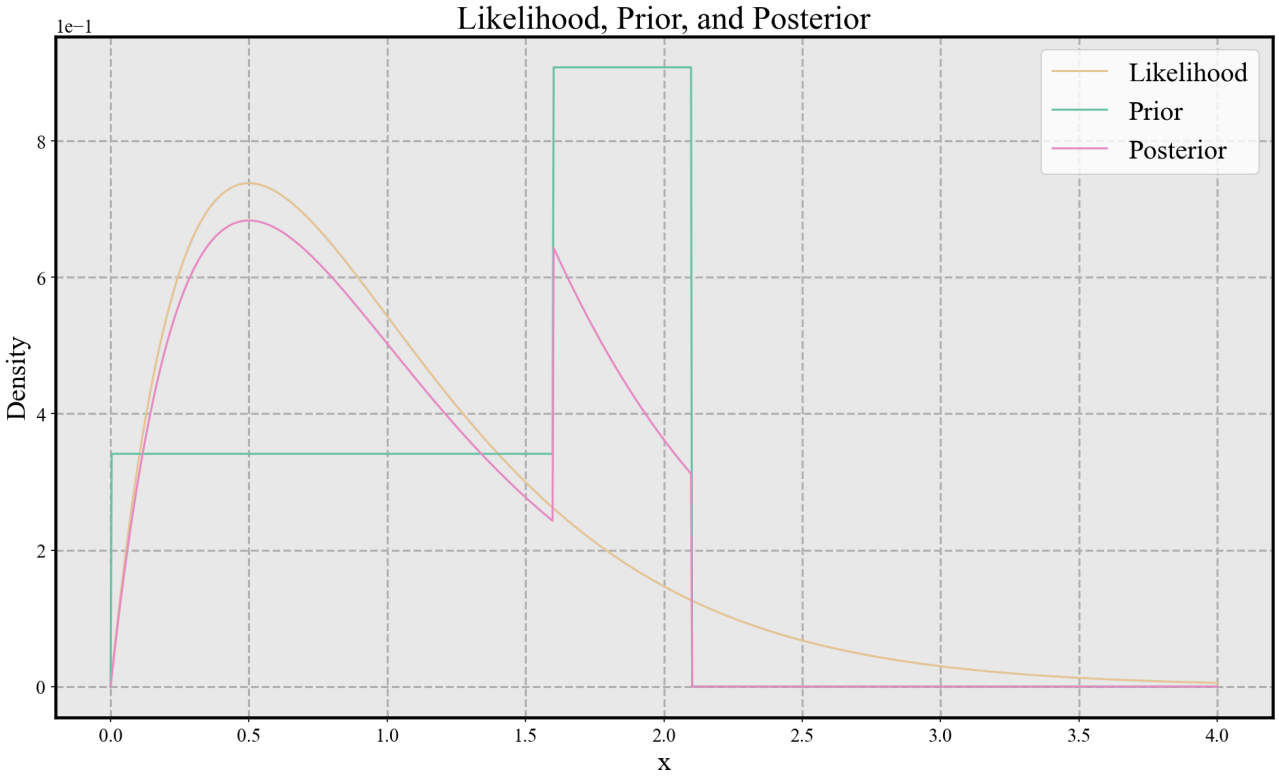

Figure 3: Prior, likelihood and posterior normalized on the range $x = [0, 4]$

2.2

I find the most likely value, $\hat{x}$, by maximizing the posterior:

$$\hat{x} = 0.50$$

The posterior at this x value is: $g(\hat{x}) = 0.68$

3

3.1

I setup a grid with an island that has a radius of 5 km and is centered at (0,0) I use a resolution of 1 meter resulting in a grid of size (10000,10000). Initializing a crab at position (3600, -2000) and

simulating 200 steps the trajectory of the crab is shown as single points on figure 4.

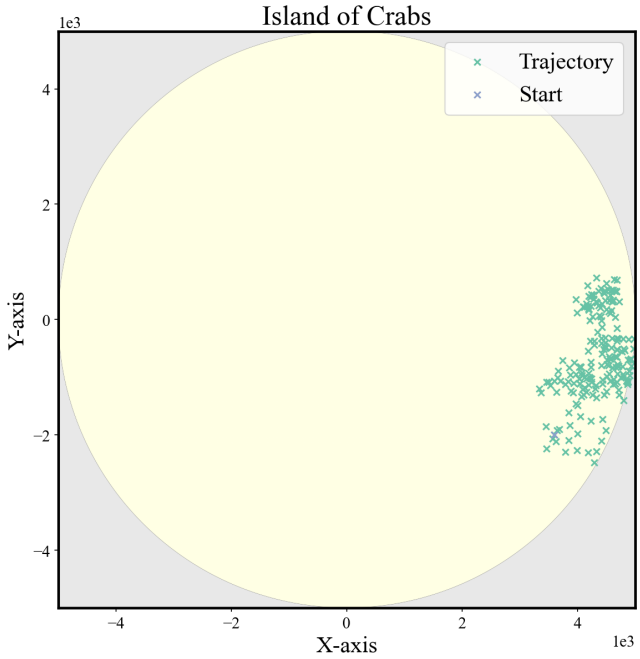

Figure 4: Movement of a single crab over 200 days

3.2

I plot the distribution of travel distances for 501 experiments of the same initialization. I show the distribution with the required specifications in figure 5.

3.3

I implement the battle algorithm explained in the problem set where larger crabs consume smaller crabs, if they are within 175 m, with some probability. The outcome of the battles are determined by the mass of the crabs. I compare M_i with M_j and calculate the ratio: $\frac{M_i^2}{M_i^2+M_j^2}$ if M_i is the smallest. This is the probability that M_i will be absorbed by M_j. I draw a uniform number between 0 and 1 and compare to the ratio. This determines the outcome.

I compare the distances of all crabs and sort such that I can resolve closest battles first. I set up 6000 experiments of the 20 starting locations and record the mass of the largest crab and the number of crabs alive after 200 time steps. I find that the starting positions is in kilometers and I rewrite this to meters.

In fig. 6 I show the number of crabs alive after 200 simulated moves.

3.4

In fig. 7 I show that the most likely mass of the largest crab is 4 kg after 200 simulation steps.

3.5

I use the same algorithm to find the number of days until there is 10 crabs left using a while loop instead of setting the number of days. The distribution is shown in figure 8. I calculate the bounds as:

$$lower = (100 - 68.27)/2$$

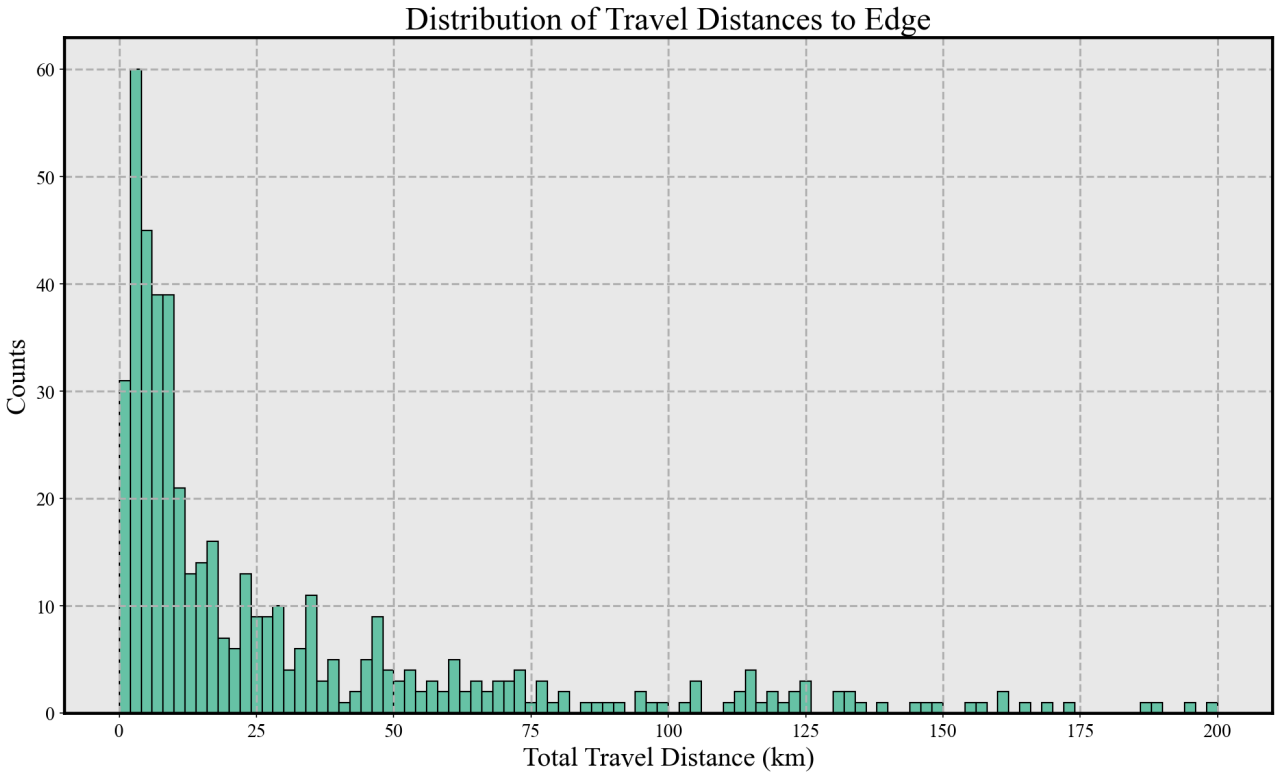

Figure 5: For each experiment I calculate the total travelled distance accounting for the smaller step when reaching the boundary. I stop when the crab reaches the edge

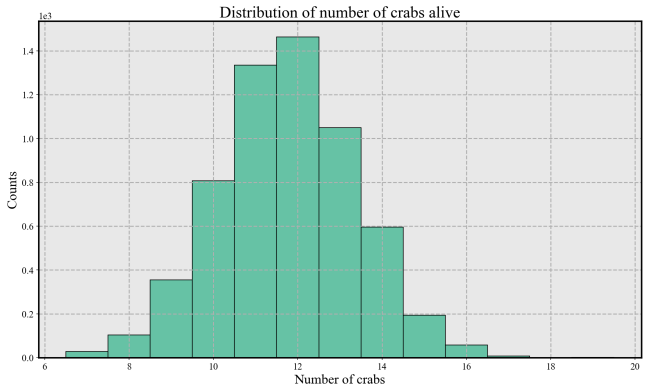

Figure 6: After 200 simulated time steps the most likely number of crabs remaining is 12.

$$upper = (100 + 68.27)/2$$

I use np.percentile(distribution, bound) to find the values and plot them.

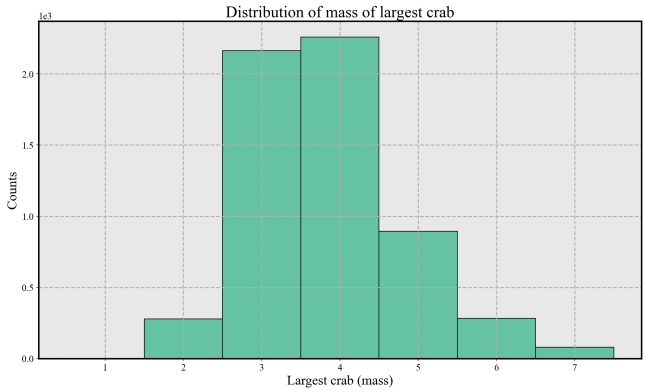

Figure 7: The mass of the largest crab after 200 simulated time steps is most likely 4 kg.

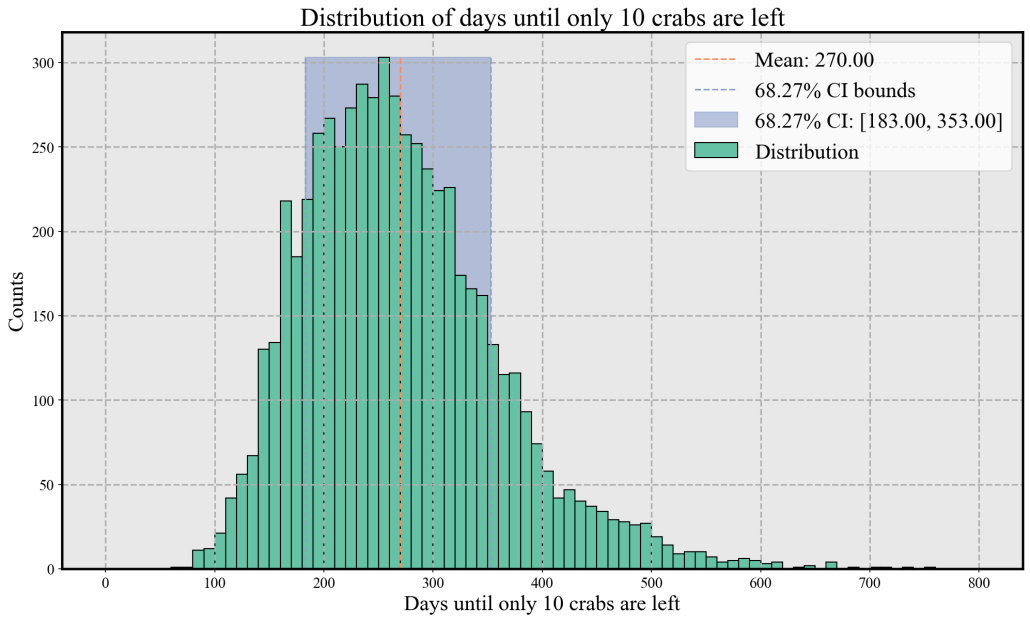

Figure 8: I plot the distribution and the upper and lower bounds for the 1σ confidence interval.